

Data Analytics (DA)

Module -3

MySQL Assignment 2 -Querying Data

Tasks

Clause & Operators:

1. DISTINCT VALUES:

* A query to retrieve distinct salaries from the Employees table.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee' selected. The main editor window shows a query titled 'DDL EMPLOYEE_DB' with the following SQL code:

```
-- 1. DISTINCT SALARY
-- A query to retrieve distinct salaries from the Employees table.
SELECT DISTINCT salary
FROM employees;
```

The query results are displayed in a table with the following data:

salary
80000.00
80000.00
40000.00
70000.00
80000.00
80000.00
80000.00
70000.00
80000.00
200000.00

The bottom panel shows the 'Action Output' tab with the following log:

#	Time	Action	Message	Duration / Rows
1	20:57:07	USE employee	0 rows affected	0:00 sec
2	20:57:14	INSERT INTO departments (Department_ID, Department_name) VALUES (1, 'Software Development'), (2, 'Marketing')	12 rows affected Records: 12 Duplicates: 0 Warnings: 0	0:156 sec
3	20:57:48	INSERT INTO locations (Location_ID, Location_name) VALUES (1, 'Bangalore'), (2, 'Hyderabad'), (3, 'Pune')	4 rows affected Records: 4 Duplicates: 0 Warnings: 0	0:138 sec
4	20:58:57	INSERT INTO employees (Employee_ID, Employee_name, Gender, Age, Hire_date, Designation, Department_ID, Location_ID)	Error Code: 1054 Unknown column 'Department_ID' in field list	0:000 sec
5	20:59:24	INSERT INTO employees (Employee_ID, Employee_name, Gender, Age, Hire_date, Designation, Department_ID, Location_ID)	30 rows affected Records: 30 Duplicates: 0 Warnings: 0	0:187 sec
6	20:59:42	SELECT distinct salary from employees LIMIT 5, 1000	11 rows returned	0:000 sec - 0:002 sec

2. ALIAS (AS):

* Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_salary"

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- 2. ALIAS (AS):  
-- * Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_salary", respectively  
SELECT Age as Employee_Age,  
       Salary as Employee_salary  
FROM employees
```

The query results are displayed in a table with the following columns: Employee_Age and Employee_salary. The results are as follows:

Employee_Age	Employee_salary
27	60000.00
21	80000.00
26	40000.00
29	70000.00
23	10000.00
28	65000.00
25	90000.00
27	20000.00
30	15000.00
29	80000.00
28	60000.00
26	98000.00
23	75000.00
30	62000.00
27	83000.00

The Action Output pane at the bottom shows the execution of the query, indicating that 30 rows were returned.

3. WHERE CLAUSE & OPERATORS:

* Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.

* Find the employee whose designation is missing and fill it with "Data Scientist".

The screenshot shows the MySQL Workbench interface with the following components:

- Navigation Panel:** Shows the database structure with 'employees' selected under the 'employees_db' schema.
- SQL Editor:** Contains the following queries:

```
1.66 * set sql_safe_updates=0;
1.67 * update employees
1.68 * set Designation="Data Scientist"
1.69 * where Designation is NULL;
1.70 * select * from employees;
```
- Result Grid:** Displays the output of the queries. The first query shows the result of the update operation, and the second query shows the list of employees with their details (Employee_ID, Employee_name, Gender, Age, Hire_date, Designation, Salary, Departments_ID, Location_ID).
- Action Output:** Shows the execution details of the queries, including the time taken and the number of rows affected.

Employee_ID	Employee_name	Gender	Age	Hire_date	Designation	Salary	Departments_ID	Location_ID
1001	Rajesh Singh	M	27	2015-01-01	Data Analyst	60000.00	3	4
1002	Rajesh Singh	M	31	2015-01-01	Network Engineer	80000.00	12	1
1003	Arunima Singh	F	28	2015-01-01	Customer Support Executive	40000.00	10	2
1004	Rajesh Singh	F	28	2015-01-01	Data Scientist	70000.00	8	3
1005	Anil Chaudhary	F	29	2015-01-01	Business Development Executive	50000.00	11	1
1006	Dr. Ravi	M	38	2015-01-01	UI Developer	60000.00	8	2
1007	Anushka Singh	F	33	2016-01-01	Marketing Manager	90000.00	2	3
1008	Dr. Ravi	F	27	2016-01-01	Graphic Designer	70000.00	8	4
1009	Ravi Singh	M	30	2016-01-01	Sales Executive	50000.00	11	7
1010	Anil Chaudhary	M	29	2016-01-01	Software Administrator	80000.00	12	4
1011	Suman Patel	F	28	2016-01-01	Marketing Analyst	60000.00	2	1
1012	Ravi Singh	F	28	2016-01-01	Operations Manager	90000.00	12	2
1013	Anil Chaudhary	M	33	2017-01-01	Customer Success Manager	70000.00	10	3
1014	Ravi Singh	M	30	2017-01-01	Supply Chain Analyst	60000.00	10	2
1015	Ravi Singh	M	27	2017-01-01	Financial Analyst	80000.00	9	1

Sorting and Grouping Data:

1. ORDER BY:

- Find employees sorted by department ID in ascending order and salary in descending order.

The screenshot displays the MySQL Workbench interface. The 'Query' tab is active, showing a SQL query that sorts employees by department ID in ascending order and salary in descending order. The 'Results' pane shows the output of the query, which is a table with columns: Employee_ID, Employee_name, Gender, Age, Hire_date, Designation, Salary, Departments_ID, and Location_ID. The 'Messages' pane at the bottom shows the execution progress of the query.

Query:

```
-- Sorting and Grouping Data:
-- Find employees sorted by department ID in ascending order and salary in descending order.
select * from emp02years
order by departments_ID asc, salary desc
```

Result:

Employee_ID	Employee_name	Gender	Age	Hire_date	Designation	Salary	Departments_ID	Location_ID
3020	Ashish Patel	M	33	2018-03-01	Software Engineer	80000.00	1	1
3067	Anushka Singh	F	32	2018-01-01	Marketing Manager	90000.00	2	3
3022	Anika Paul	F	30	2018-09-20	Public Relations Specialist	70000.00	2	2
3011	Saami Patel	F	28	2018-09-20	Marketing Analyst	60000.00	2	1
3025	Arun Tyengar	P	31	2019-01-20	Data Scientist	100000.00	3	4
3026	Viveen Singh	M	28	2019-03-20	Business Analyst	70000.00	3	2
3061	Vishen Singh	M	27	2015-01-20	Data Analyst	60000.00	3	6
3028	Zoya Khan	F	31	2017-07-09	Legal Counsel	100000.00	4	6
3022	Aditya Loringar	P	28	2019-07-10	HR Specialist	60000.00	4	4
3010	Shikha Patel	F	29	2018-09-21	Talent Acquisition Specialist	50000.00	4	6
3024	Aryan Shetty	M	27	2018-11-01	Product Manager	90000.00	5	1
3030	Sanku Loya	F	28	2019-11-20	Project Manager	80000.00	5	1
3027	Ananya Paul	F	32	2018-09-20	Content Writer	60000.00	6	3
3028	Ananya Kapoor	F	26	2019-07-09	Event Coordinator	60000.00	6	1
3015	Shweta Kumar	M	27	2017-05-20	Financial Analyst	25000.00	7	1
3016	Shikha Patel	M	28	2018-09-21	Marketing Analyst	60000.00	7	4

Messages:

- 11: 25:47:28 update employees set Designation='Data Scientist' where Designation is NULL
- 12: 25:49:30 select * from employees LIMIT 0, 1000
- 13: 25:50:18 set Def_val=location=1
- 14: 25:55:30 select * from employees order by departments_ID, Salary desc LIMIT * from employees
- 15: 25:57:33 select * from employees order by departments_ID, Salary desc LIMIT * from employees
- 16: 25:58:51 select * from employees order by departments_ID asc, Salary desc LIMIT 0, 1000
- 17: 25:59:30 select * from employees order by departments_ID asc, Salary desc LIMIT 0, 1000

2. LIMIT:

- Display the first 5 employees hired in the year 2018.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
179 -- LIMIT: Display the first 5 employees hired in the year 2018.
180 select * from employees
181 where year(Hire_date)=2018
182 order by Hire_date desc
183 limit 5;
```

The result grid displays the following data:

Employee_ID	Employee_name	Gender	Age	Hire_date	Designation	Salary	Departments_ID	Location_ID
9004	Arjun Shetty	M	27	2018-12-01	Product Manager	8000.00	3	1
9003	Anika Paul	F	30	2018-09-28	Public Relations Specialist	7000.00	2	2
9002	Adrius Longan	F	28	2018-07-18	HR Specialist	6000.00	4	4
9001	Aditi Kapoor	M	26	2018-05-28	Finance Analyst	5000.00	7	3
9005	Aarav Nair	M	22	2018-03-05	Software Engineer	9000.00	1	1

The table structure for 'employees' is also shown:

Column	DataType	PK
Employee_ID	int	PK
Employee_name	varchar(45)	
Gender	enum('M', 'F')	
Age	int	
Hire_date	date	
Designation	varchar(35)	
Salary	decimal(8,2)	
Departments_ID	int	
Location_ID	int	

3.AGGREGATE FUNCTIONS:

- Calculate the sum of all salaries in the Finance department.
- Find the minimum age among all employees.

The screenshot shows the MySQL Workbench interface with two queries in the 'SUBQUERYLIVE' window:

```
193
194
195 select min(Age)
196 as Minimum_Age
197 from employees;
```

The result grid shows the following data:

Column	Value
Total_Salary_Finance	85000.00

The table structure for 'employees' is also shown:

Column	DataType	PK
Employee_ID	int	PK
Employee_name	varchar(45)	
Gender	enum('M', 'F')	
Age	int	
Hire_date	date	
Designation	varchar(35)	
Salary	decimal(8,2)	
Departments_ID	int	
Location_ID	int	

The action output shows the following results:

Time	Action	Message	Duration / Fetch
1 13:51:27	USE Employee	0 rows(s) affected	0.000 sec
2 13:52:03	Select sum(Salary) as Total_Salary_Finance from employees where Designation='Finance Analysts' LIMIT 0, 1000	1 row(s) returned	0.047 sec / 0.000 sec
3 13:52:03	Select min(Age) as Minimum_Age from employees LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
4 13:53:01	Select sum(Salary) as Total_Salary_Finance from employees where Designation='Finance Analyst' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
5 13:53:01	Select min(Age) as Minimum_Age from employees LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

college_database
companydb
ecommercedb
employee
departments
employees
locations
Views
Stored Procedures
Functions
employee_db
hospital
online_bookstore
sakila
sales
sales_analytics
Administration Schemas

Table: employees

Columns:

Employee_ID int PK
Employee_name varchar(45)
Gender enum('M', 'F')
Age int
Hire_date date
Designation varchar(45)
Salary decimal(8, 2)
Departments_ID int
Location_ID int

DDL EMPLOYEE_DB

```

194 where Departments_name='Finance');
195
196 select min(Age)
197 as Minimum_Age
198 from employees;

```

Result Grid

Total_Salary_Finance
170000.00

Result 7 x Result 8

Output

#	Time	Action	Message	Duration / Fetch
1	08:38:51	select min(Age) as Minimum_Age from employees LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2	08:39:15	Select sum(Salary) as Total_Salary_Finance from employees where Departments_ID=(select Departments_ID from de...	1 row(s) returned	0.000 sec / 0.000 sec
3	08:39:15	select min(Age) as Minimum_Age from employees LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

4. GROUP BY:

- List the maximum salary for each location.
- Calculate the average salary for each designation containing the word 'Analyst'.

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

college_database
companydb
ecommercedb
employee
departments
employees
locations
Views
Stored Procedures
Functions
employee_db
hospital
online_bookstore
sakila
sales
sales_analytics
Administration Schemas

Table: employees

Columns:

Employee_ID int PK
Employee_name varchar(45)
Gender enum('M', 'F')
Age int
Hire_date date
Designation varchar(45)
Salary decimal(8, 2)
Departments_ID int
Location_ID int

DDL EMPLOYEE_DB

```

201 -- List the maximum salary for each location.
202 -- Calculate the average salary for each designation containing the word 'Analyst'.
203 select max(Salary) as Max_Salary
204 from employees
205 Group by Location_ID;

```

Result Grid

Max_Salary
95000.00
95000.00
90000.00
100000.00

Result 5 x

Output

#	Time	Action	Message	Duration / Fetch
1	08:38:51	select min(Age) as Minimum_Age from employees LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2	08:39:15	Select sum(Salary) as Total_Salary_Finance from employees where Departments_ID=(select Departments_ID from de...	1 row(s) returned	0.000 sec / 0.000 sec
3	08:39:15	select min(Age) as Minimum_Age from employees LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
4	08:45:54	Select max(Salary) as Max_Salary from employees Group by Location_ID LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Navigation

Filter objects

college_database

companydb

ecommercedb

employee

departments

employees

locations

Views

Stored Procedures

Functions

employee_db

hospital

online_bookstore

salila

sales

sales_analytics

Administration

Schemas

Table: employees

Columns:

Employee_ID

Employee_name

Gender

Age

Hire_date

Designation

Departments_ID

Location_ID

int PK

varchar(255)

enum('M', 'F')

int

date

varchar(255)

int

int

ASSIGNMENT2

DDL EMPLOYEE_DB

Limit to 1000 rows

207

Select

208

Designation, avg(salary) as Avg_Salary

209

from employees

210

where Designation Like "Analyst"

211

Group by Designation;

Result Grid

Filter Rows

Export

Wrap Cell Content

Result Grid

Form Editor

Field Types

Query State

Read Only

Output

Action Output

Time

Action

Message

Duration / Fetch

1 08:38:51 select min(Age) as Minimum_Age from employees LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec

2 08:39:15 select sum(Salary) as Total_Salary_Finance from employees where Departments_ID= (select Departments_ID from de... 1 row(s) returned 0.000 sec / 0.000 sec

3 08:39:15 select min(Age) as Minimum_Age from employees LIMIT 0, 1000 1 row(s) returned 0.016 sec / 0.000 sec

4 08:45:54 select max(Salary) as Max_Salary from employees Group by Location_ID LIMIT 0, 1000 4 row(s) returned 0.000 sec / 0.000 sec

5 08:51:36 select Designation, avg(salary) as Avg_Salary from employees where Designation Like "%Analyst%" Group by Designati... 7 row(s) returned 0.000 sec / 0.000 sec

5. HAVING:

- Find departments with less than 3 employees.
- Find locations with female employees whose average age is below 30.

The screenshot shows the SQL Developer interface with a query window titled 'ASSIGNMENT2'. The query is as follows:

```
-- * Find locations with female employees whose average age is below 30.
216 Select Departments_ID, count(*) as Employee_Count
217 from employees
218 Group by Departments_ID
219 Having Count(*) < 3
```

The 'Result Grid' shows the following data:

Departments_ID	Employee_Count
1	1
5	2
6	2
7	2
9	1
11	2
13	1

The 'Table: employees' structure is shown below:

Columns:

- Employee_ID (int PK)
- Employee_name (varchar(10))
- Gender (enum('M', 'F'))
- Age (int)
- Hire_date (date)
- Designation (varchar(10))
- Salary (decimal(10, 2))
- Departments_ID (int)
- Location_ID (int)

The 'Action Output' pane shows the execution of the query, with the final result being 7 rows returned.

The screenshot shows the SQL Developer interface with a query window titled 'SUBQUERYLIVE'. The query is as follows:

```
221 Gender,
222 avg(age) as Avg_age
223 from employees
224 Group by Gender, Location_ID
225 Having Avg_age < 30 and Gender = 'F';
```

The 'Result Grid' shows the following data:

Location_ID	Gender	Avg_age
2	F	27.3333
1	F	26.7500
4	F	29.2000

The 'Table: employees' structure is shown below:

Columns:

- Employee_ID (int PK)
- Employee_name (varchar(10))
- Gender (enum('M', 'F'))
- Age (int)
- Hire_date (date)
- Designation (varchar(10))
- Salary (decimal(10, 2))

The 'Action Output' pane shows the execution of the query, with the final result being 3 rows returned.

Joins:

1. INNER JOIN:

- List employee names, their designations, and department names where employees are assigned to a department.

The screenshot shows a MySQL Workbench window with a query editor and a result grid. The query is an INNER JOIN between the 'employees' and 'departments' tables. The result grid displays 17 rows of data, showing employee names, designations, and department names.

```
230 d.Departments_Name
231 from employees as e
232 inner join
233 Departments as d
234 on e.Departments_ID=d.Departments_ID;
```

Employee_Name	Designation	Departments_Name
Anvi Chaudhary	Business Development Executive	Business Development
Kaan Desai	Sales Executive	Business Development
Ananya Paul	Content Writer	Content Creation
Anaya Kapoor	Event Coordinator	Content Creation
Aradhya Iyer	Customer Support Executive	Customer Support
Amar Ras	Customer Success Manager	Customer Support
Vhaan Mohan	Supply Chain Analyst	Customer Support
Vhaan Singh	Data Analyst	Data Science
Avni Iyengar	Data Scientist	Data Science
Vivaan Singh	Business Analyst	Data Science
Kara Malhotra	Data Scientist	Design
Dhruv Shetty	UI Developer	Design
Diya Jha	Graphic Designer	Design
Ishaan Kumar	Financial Analyst	Finance
Advik Kapoor	Finance Analyst	Finance

2. LEFT JOIN:

- List all departments along with the total number of employees in each department, including departments with no employees.

The screenshot shows a MySQL Workbench window with a query editor and a result grid. The query is a LEFT JOIN between the 'departments' and 'employees' tables, grouped by department name. The result grid displays 12 rows of data, showing department IDs, names, and total employee counts.

```
242 from departments as d
243 left join
244 employees as e
245 on e.Departments_ID=d.Departments_ID
246 Group by departments_name;
```

Departments_ID	departments_name	Total_Employees
1	Software Development	1
2	Marketing	3
3	Data Science	3
4	Human Resources	3
5	Product Management	2
6	Content Creation	2
7	Finance	2
8	Design	3
9	Research and Development	1
10	Customer Support	3
11	Business Development	2
12	IT	4

3. RIGHT JOIN:

- Display all locations along with the names of employees assigned to each location.

If no employees are assigned to a location, display NULL for employee name.

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

ASSIGNMENT2 DDL EMPLOYEE_DB

Limit to 1000 rows

251 e.Employee_name
252 from employees as e
253 right join
254 Locations as l
255 on e.Location_ID=l.Location_ID;

Result Grid

Location_name	Employee_name
Bangalore	Aaradhyia Iyer
Bangalore	Dhruv Shetty
Bangalore	Myra Verma
Bangalore	Vivian Mohan
Bangalore	Kabir Nair
Bangalore	Anika Paul
Bangalore	Vivian Singh
Bangalore	Arjun Kumar
Chennai	Reyansh Singh
Chennai	Anvi Choudhary
Chennai	Saanvi Patel
Chennai	Ishaan Kumar
Chennai	Aarav Nair
Chennai	Aryan Shetty
Chennai	Anaya Kapoor

Result 23 x

Output

#	Time	Action	Message	Duration / Fetch
1	09:47:42	Select count(*) Employee_ID as Total_Employees, d.Departments_ID from employees as e left join departments as d on...	13 row(s) returned	0.000 sec / 0.000 sec
2	09:55:28	Select l.Location_name, e.Employee_name from employees as e right join Locations as l on e.Location_ID=l.Location_ID	30 row(s) returned	0.047 sec / 0.000 sec

Table: employees

Columns:

- Employee_ID int PK
- Employee_name varchar(100)
- Gender enum('M', 'F')
- Age int
- Hire_date date
- Designation varchar(100)
- Salary decimal(10, 2)
- Departments_ID int
- Location_ID int

4. CROSS JOIN

- Show all possible combinations of departments and locations.

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

ASSIGNMENT2 DDL EMPLOYEE_DB SQL File 4

Limit to 1000 rows

260 d.Departments_name,
261 l.Location_name
262 from departments as d
263 cross join
264 locations as l;

Result Grid

Departments_name	Location_name
Business Development	Pune
Business Development	Hyderabad
Business Development	Chennai
Business Development	Bangalore
Content Creation	Pune
Content Creation	Hyderabad
Content Creation	Chennai
Content Creation	Bangalore
Customer Support	Pune
Customer Support	Hyderabad
Customer Support	Chennai
Customer Support	Bangalore
Data Science	Pune
Data Science	Hyderabad

Result 24 x

Output

#	Time	Action	Message	Duration / Fetch
1	09:47:42	Select count(*) Employee_ID as Total_Employees, d.Departments_ID from employees as e left join departments as d on...	13 row(s) returned	0.000 sec / 0.000 sec
2	09:55:28	Select l.Location_name, e.Employee_name from employees as e right join Locations as l on e.Location_ID=l.Location_ID	30 row(s) returned	0.047 sec / 0.000 sec
3	10:05:02	Select d.Departments_name, l.Location_name from departments as d cross join locations as l on d.Departments_ID=l...	Error Code: 1054. Unknown column 'l.Departments_ID' in 'on clause'	0.000 sec
4	10:05:52	Select d.Departments_name, l.Location_name from departments as d cross join locations as l LIMIT 0, 1000	52 row(s) returned	0.250 sec / 0.000 sec

Table: locations

Columns:

- Location_ID int AI PK
- Location_name varchar(30)

5. SELF JOIN:

- Show pairs of employees working in the same department, excluding self-pairs.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```

276 from employees e1
277 inner join
278 employees e2
279 on e2.Departments_ID=e1.Departments_ID
280 AND e2.employee_ID>e1.employee_ID;

```

The result grid displays the following data:

Emp_ID1	Emp_name1	Emp_ID2	Emp_Name2	Departments_ID
5001	Vhaan Singh	5025	Avni Iyengar	3
5001	Vhaan Singh	5026	Vhaan Singh	3
5002	Reyansh Singh	5010	Atharv Yadav	12
5002	Reyansh Singh	5017	Kabir Nair	12
5002	Reyansh Singh	5029	Aryun Kumar	12
5003	Aaradhya Iyer	5013	Arnav Rao	10
5003	Aaradhya Iyer	5014	Vhaan Mohan	10
5004	Kiara Malhotra	5006	Dhruv Shetty	8
5004	Kiara Malhotra	5008	Dya Jha	8
5005	Arvi Chaudhary	5009	Raan Desai	11
5006	Dhruv Shetty	5008	Dya Jha	8
5007	Anushka Singh	5011	Saanvi Patel	2

The bottom pane shows the execution log with three successful queries.

Windows function

- Write a window function query to rank employees by salary using rank().

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```

285 Employee_ID,
286 Employee_name,
287 Salary,
288 Rank() over(order by salary desc) as Rank_employees
289 from employees;

```

The result grid displays the following data:

Employee_ID	Employee_name	Salary	Rank_employees
5016	Zoya Khan	100000.00	1
5025	Avni Iyengar	100000.00	1
5012	Myra Verma	95000.00	3
5024	Aryan Shetty	95000.00	3
5007	Anushka Singh	90000.00	5
5020	Aarav Nair	90000.00	5
5030	Sara Iyer	90000.00	5
5015	Ishaan Kumar	85000.00	8
5021	Advik Kapoor	85000.00	8
5002	Reyansh Singh	80000.00	10
5010	Atharv Yadav	80000.00	10
5017	Kabir Nair	80000.00	10

The bottom pane shows the execution log with two successful queries.

- Write a window function query to rank employees by salary within each department using DENSE_RANK()

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

college_database

companydb

ecommercedb

employee

Tables

departments

employees

locations

Views

Stored Procedures

Functions

employee_db

hospital

online_bookstore

sakila

sales

Administration Schemas

Information

Table: employees

Columns:

Employee_ID int PK

Employee_name varchar(45)

Gender enum('M', 'F')

Age int

Hire_date date

Designation varchar(45)

Salary decimal(10, 2)

Departments_ID int

Location_ID int

ASSIGNMENT2 SUBQUERYLIVE

Limit to 1000 rows

294 Employee_ID,

295 Employee_name,

296 Salary,

297 Dense_Rank() over(partition by Departments_ID Order by salary desc) as Rank_employeesby_Departments

298 from employees;

Result Grid

Employee_ID	Employee_name	Salary	Rank_employeesby_Departments
5020	Aarav Nair	90000.00	1
5007	Anushka Singh	90000.00	1
5023	Anika Paul	70000.00	2
5011	Saami Patel	60000.00	3
5025	Aam Iyengar	100000.00	1
5026	Vivaan Singh	75000.00	2
5001	Vhaan Singh	60000.00	3
5016	Zoya Khan	100000.00	1
5022	Aadhya Iyengar	60000.00	2
5019	Ishika Patel	55000.00	3
5024	Aryan Shetty	95000.00	1
5030	Sara Iyer	90000.00	2
5027	Ananya Paul	60000.00	1

Action Output

Time	Action	Message	Duration / Fetch
1 13:08:47	select e1.Employee_ID as Emp_ID1, e1.Employee_name as Emp_name1, e2.Employee_ID as Emp_ID2, e2.Employee_...	25 row(s) returned	0.000 sec / 0.000 sec
2 13:21:45	select Employee_ID, Employee_name, Salary, Rank() over(order by salary desc) as Rank_employees from employees	30 row(s) returned	0.000 sec / 0.000 sec
3 13:24:37	select Employee_ID, Employee_name, Salary, Dense_Rank() over(partition by Departments_ID Order by salary desc) as ...	30 row(s) returned	0.000 sec / 0.000 sec
4 13:25:12	select Employee_ID, Employee_name, Salary, Dense_Rank() over(partition by Departments_ID Order by salary desc) as ...	30 row(s) returned	0.000 sec / 0.000 sec

• Write a window function query, Running total salary by department

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

college_database

companydb

ecommercedb

employee

Tables

departments

employees

locations

Views

Stored Procedures

Functions

employee_db

hospital

online_bookstore

sakila

sales

Administration Schemas

Information

Table: employees

Columns:

Employee_ID int PK

Employee_name varchar(45)

Gender enum('M', 'F')

Age int

Hire_date date

Designation varchar(45)

Salary decimal(10, 2)

Departments_ID int

Location_ID int

ASSIGNMENT2 SUBQUERYLIVE

Limit to 1000 rows

301 Employee_ID,

302 Employee_name,

303 Salary,

304 sum(salary) over(partition by Departments_ID order by Employee_ID) as Running_Total

305 from employees;

Result Grid

Employee_ID	Employee_name	Salary	Running_Total
5020	Aarav Nair	90000.00	90000.00
5007	Anushka Singh	90000.00	90000.00
5011	Saami Patel	60000.00	150000.00
5023	Anika Paul	70000.00	220000.00
5001	Vhaan Singh	60000.00	60000.00
5025	Aam Iyengar	100000.00	160000.00
5026	Vivaan Singh	75000.00	235000.00
5016	Zoya Khan	100000.00	100000.00
5019	Ishika Patel	55000.00	155000.00
5022	Aadhya Iyengar	60000.00	215000.00
5024	Aryan Shetty	95000.00	95000.00
5030	Sara Iyer	90000.00	185000.00
5027	Ananya Paul	60000.00	60000.00

Action Output

Time	Action	Message	Duration / Fetch
1 13:33:22	select Employee_ID, Employee_name, Salary, sum(salary) over(partition by Departments_ID order by Employee_ID) as ...	30 row(s) returned	0.000 sec / 0.000 sec
2 13:34:04	select e1.Employee_ID as Emp_ID1, e1.Employee_name as Emp_name1, e2.Employee_ID as Emp_ID2, e2.Employee_...	25 row(s) returned	0.000 sec / 0.000 sec
3 13:34:13	select Employee_ID, Employee_name, Salary, Rank() over(order by salary desc) as Rank_employees from employees	30 row(s) returned	0.000 sec / 0.000 sec
4 13:34:21	select Employee_ID, Employee_name, Salary, Dense_Rank() over(partition by Departments_ID Order by salary desc) as ...	30 row(s) returned	0.000 sec / 0.000 sec
5 13:34:30	select Employee_ID, Employee_name, Salary, sum(salary) over(partition by Departments_ID order by Employee_ID) as ...	30 row(s) returned	0.000 sec / 0.000 sec